

# Parallel Computing



João Luís Ferreira Sobral  
[www.di.uminho.pt/~jls](http://www.di.uminho.pt/~jls)  
jls@...

Web: Elearning

# 2ª Sessão de apoio ao projeto

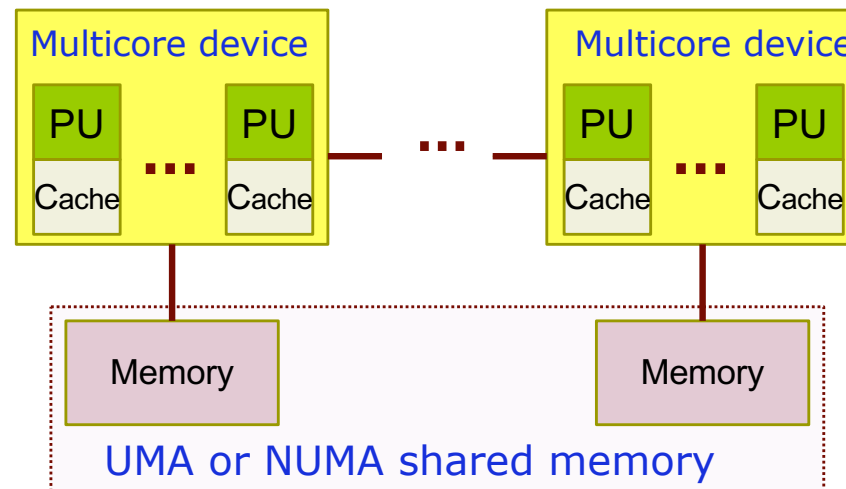
---

- ❑ 2<sup>nd</sup> Session (16h-18h, ed1 1.22 + 18h-20h ed1. 1.30)
  - Become familiarised with the case study & profiling tools
- ❑ WA1 – Goals
  - Identify opportunities to explore parallel processing
  - Develop a strategy to explore parallelism using OpenMP
- ❑ **Due date: 24/11**
  - **10% weight**
- ❑ Other dates & weights (recap):
  - Test: 9/1/2026 (40%)
  - WA final & defence: from 12 to 16/1/2026 (50%)

# Explicit parallel computing (1)

## Shared Memory (SM) parallel systems

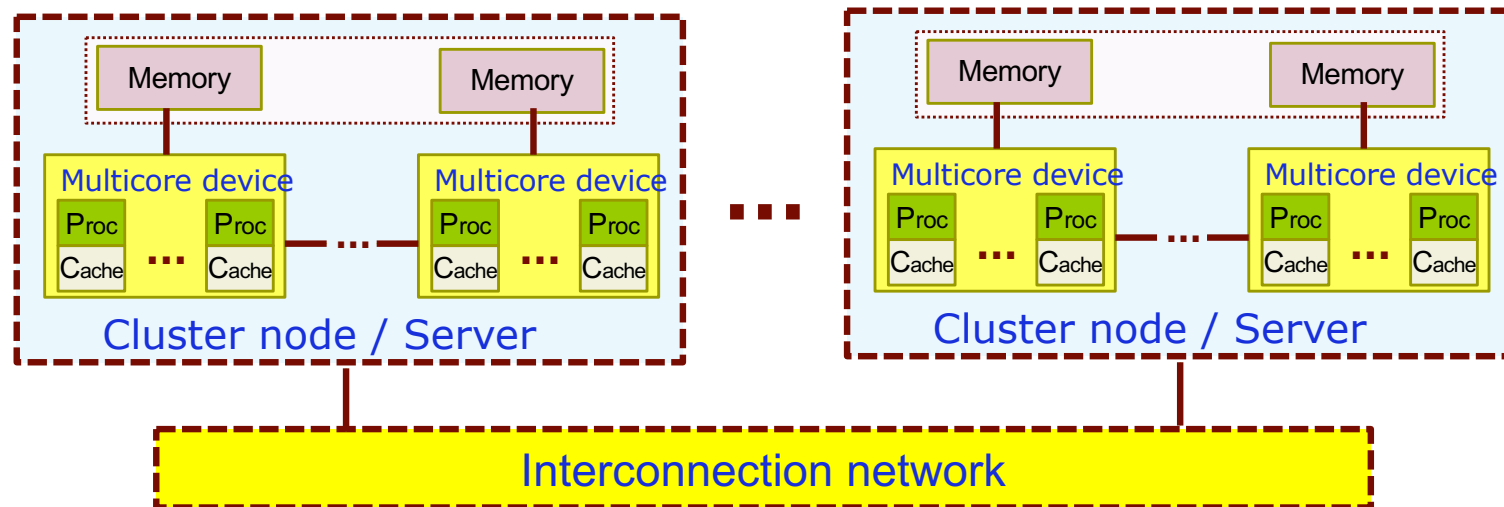
- parallelism on single or multiple devices (*same motherboard*)
  - single physical memory address space
  - each core (PU) can support multiple threads (SMT)
  - **memory bandwidth is shared by all cores**
  - **coherence of data in multiple caches?**



# Explicit parallel computing (2)

## Distributed memory parallel systems

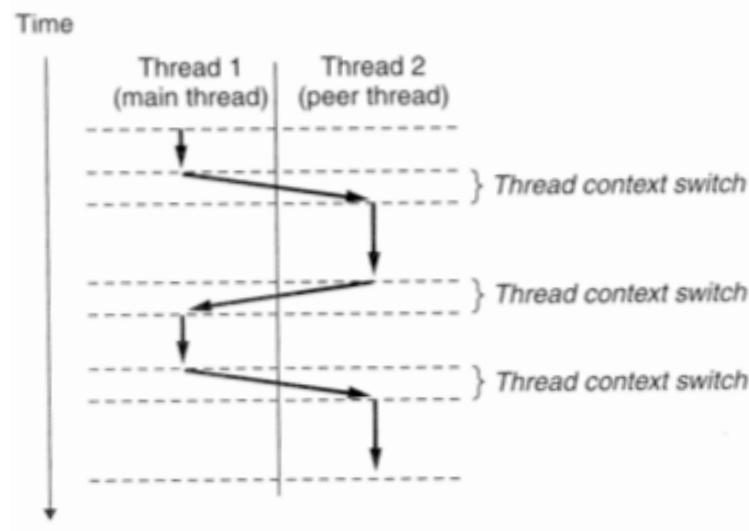
- on multiple boards (*or multiple nodes/servers*)
  - each node with its private memory space
    - **memory bandwidth is proportional to the number of nodes**



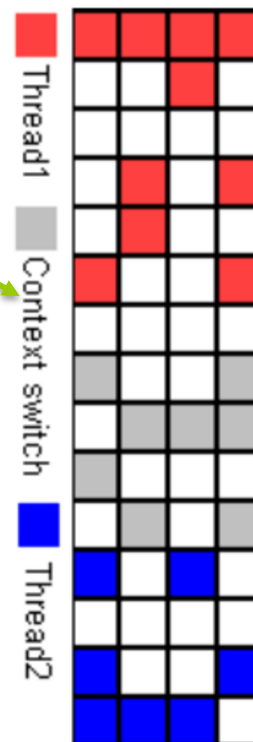
# Thread Scheduling

## (on shared memory parallel systems)

**Review:** OS thread scheduling and context switch



Superscalar  
PU (4-way)



### Thread scheduling at the Operating System (OS) level:

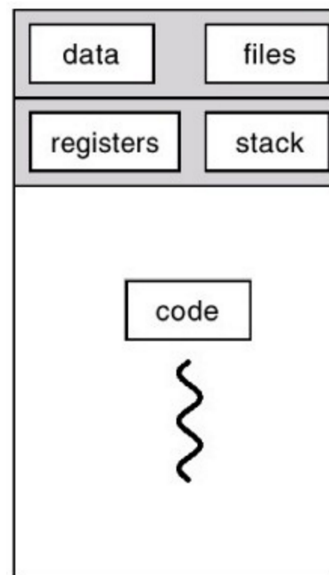
1. Interrupts execution of **Thread1** & saves **Thread1** context information (e.g., PU registers)
2. Selects the next thread to execute (**Thread2**)
3. OS restores the state of **Thread2** and gives control to it
4. **Thread2** continues execution from its previous execution point (saved Program Counter)

# Programming model: Process vs Threads

## □ Processes

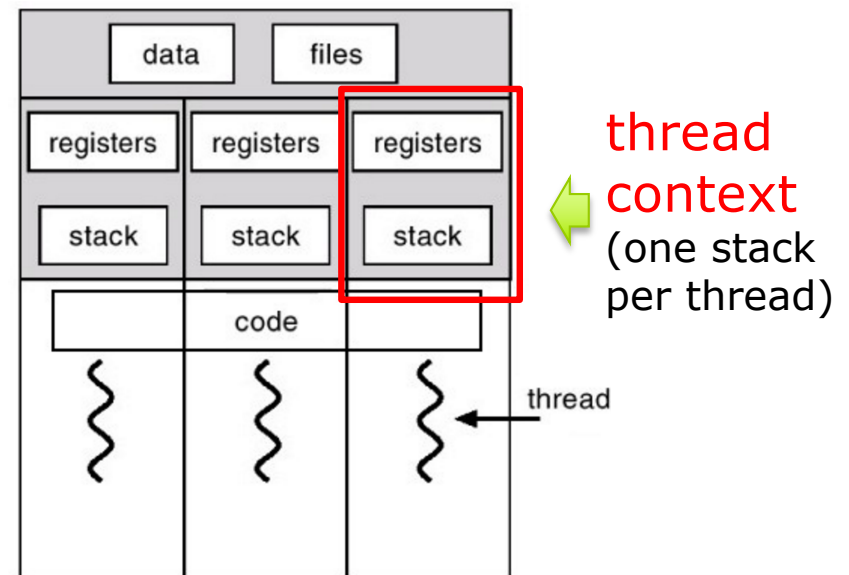
- Used for unrelated tasks
  - (e.g., a program)
- Own address space
  - Address space is protected from other process
- Switching at the kernel (OS) level

Every process has at least one thread

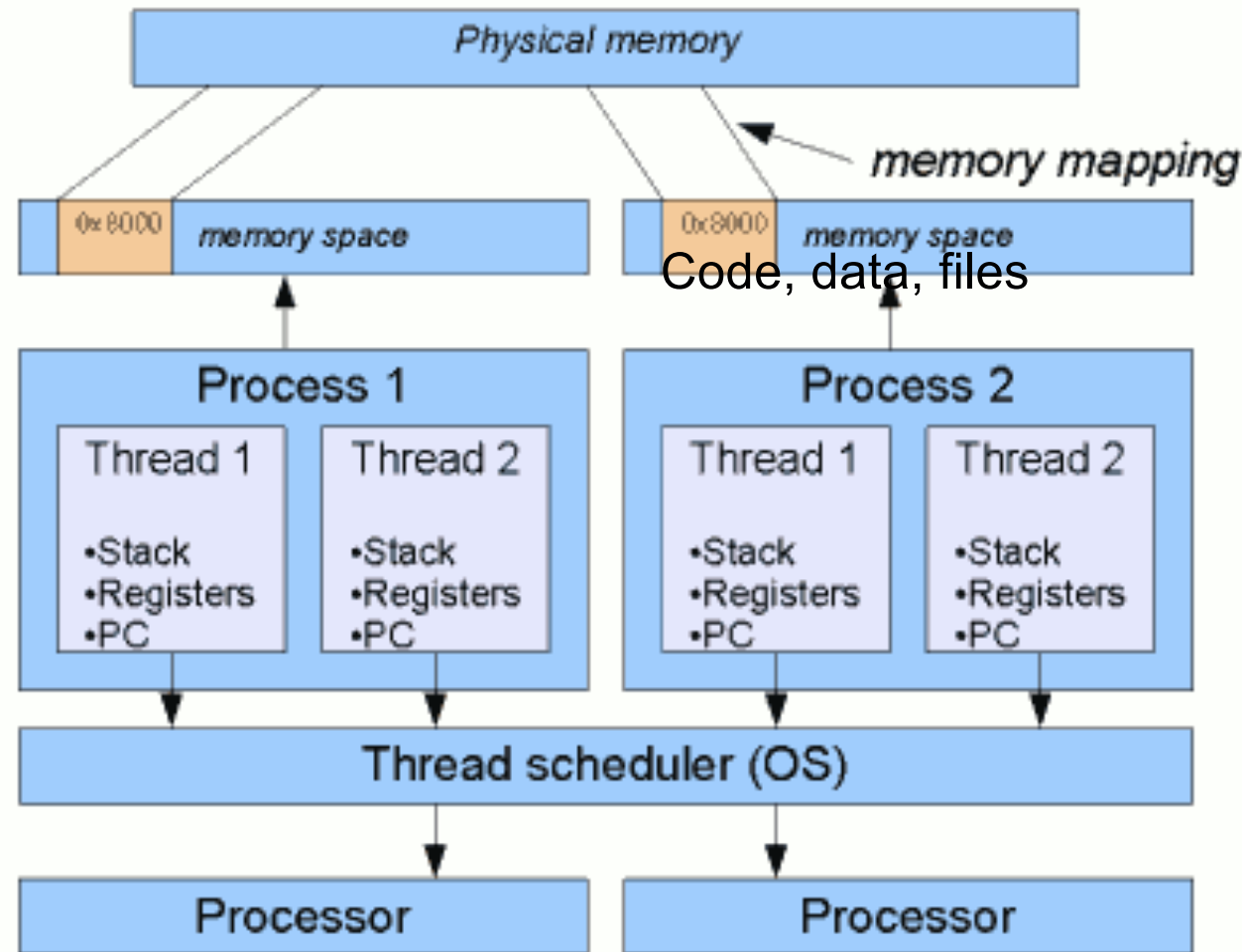


## □ Threads

- Are part from the same job
- Share address space, code, data and files
- Switching at the user or kernel level



# Thread vs Process

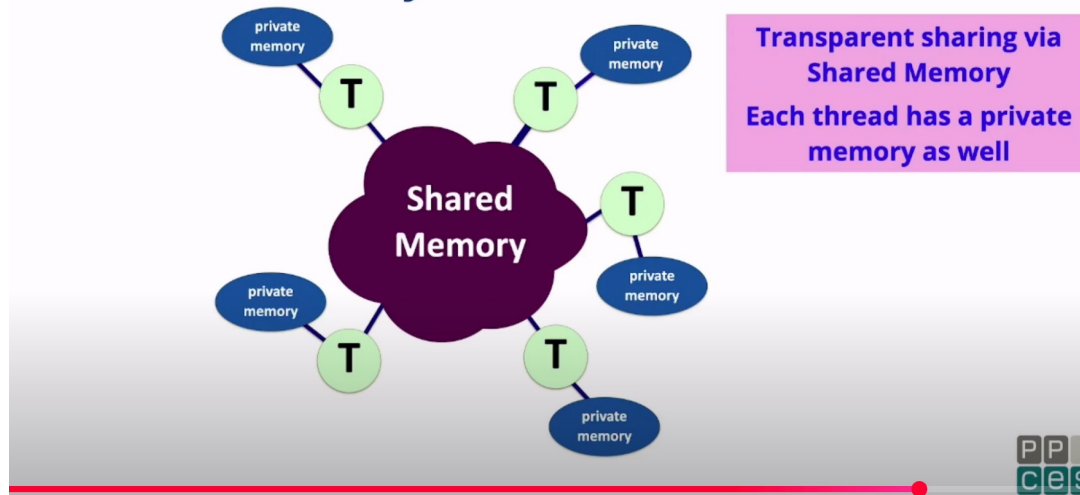


← process memory space is shared by all threads of a process

Threads from all processes are scheduled into available PUs

# Shared memory & Data races

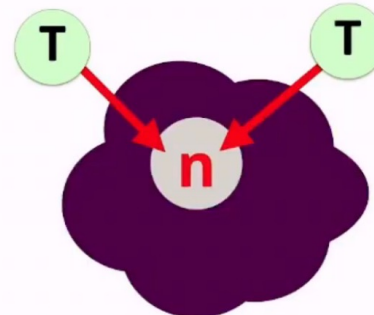
## The Shared Memory Model



- A data race **can happen** when two or more threads access (write!) to a **shared memory position**

## An Example of a Data Race

```
#pragma omp parallel shared(n)
{
    n = omp_get_thread_num();
} // End of parallel region
```



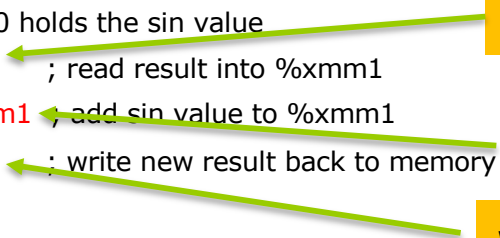
# Data races – more subtle example

- A data race **can happen** when two or more threads access (write!) to a **shared memory position**

```
5 int main(){  
6     double result={0};  
7  
8     #pragma omp parallel for shared(result)  
9     for(int i=0; i<1000000;i++) {  
10         result+=sin(i);  
11     }  
12     printf("%f",result);  
13 }
```



```
.L4:  
...  
call sin      ; after sin call, xmm0 holds the sin value  
vmovsd 8(%rsp,r12), %xmm1 ; read result into %xmm1  
vaddsd %xmm0, %xmm1, %xmm1 ; add sin value to %xmm1  
vmovsd %xmm1, 8(%rsp,r12) ; write new result back to memory  
...  
cmpl %ebx, %ebp ; i<1000000?  
jne .L4
```



Thread 0

read result

add sin

write result

Thread 1

read result

add sin

write result